

HavenWorld

Complete Build Guide — Part 4B

Phase 1 Linux: Authentication, Socket.io, RoomManager, and Moderation

Developer	Daniel Boone
Repository	github.com/dboone323/HavenWorld
Game Server	https://[your-oracle-ip].nip.io
Date	September 18, 2026
Document	Part 4B — Phase 1 Linux (Auth, Socket.io, RoomManager, Moderation)
Preceded By	Part 4A — Server Entry Point, Prisma Schema, Migrations, and Seeding
Followed By	Part 4C — Admin Routes, Invite Codes, PM2, GitHub Actions, Alpha Gate

✓ Part 1 — Foundations & Monorepo	✓ Part 2 — Database & Shared Types	✓ Part 3 — Client Scaffolding & Phaser
✓ Part 4A — Entry Point, Prisma, Migrations, Seed	► Part 4B — Auth, Socket.io, RoomManager, Moderation	○ Part 4C — Admin, Deploy, PM2, Alpha Gate

DOCUMENT CONVENTIONS

MAC Steps written on the MacBook Pro and committed to GitHub — the Oracle VM then pulls and runs the result.

LINUX Steps executed directly on the Oracle Cloud ARM VM via SSH.

Step identifiers such as **LINUX-1-C1** are sequential within this document and continue from Part 4A.

Table of Contents

Section 8 — Authentication System	3
8.1 Auth Routes Overview & Security Architecture	3
8.2 Registration Endpoint (LINUX-1-C1)	4
8.3 Email Verification Endpoint (LINUX-1-C2)	8
8.4 Login Endpoint (LINUX-1-C3)	9
8.5 Token Refresh, Logout (LINUX-1-C4, C5)	12
8.6 Users, Rooms, Friends, Reports Routes (LINUX-1-C6 through C9)	15
Section 9 — RoomManager Service	24
9.1 In-Memory Game State Design (LINUX-1-D1)	24
Section 10 — Moderation Service	28
10.1 Profanity Filter & Rate Limiter (LINUX-1-E1)	28
Section 11 — Socket.io Real-Time Handlers	30
11.1 Socket Auth Middleware (LINUX-1-F1)	30
11.2 auth:join Handler (LINUX-1-F2)	32
11.3 player:move, chat:send, avatar:update, disconnect (LINUX-1-F3)	37
11.4 Testing the Full Socket Flow (LINUX-1-F4)	44
11.5 Part 4B Completion Checklist	45

Section 8 — Authentication System	3
Continuation Note	47

Section 8: Authentication System MAC then LINUX

All auth route files are written on the MacBook Pro and pushed to GitHub. The Oracle VM pulls the updated repository and runs the compiled output. This section covers the complete REST-based authentication surface: registration with email verification and invite-code gating, JWT access + refresh token issuance, Redis-backed token allowlisting, and clean logout with immediate revocation.

8.1 Auth Routes Overview & Security Architecture

The auth system is designed to defend against the most common web attack vectors — XSS, CSRF, replay attacks, credential stuffing, and brute-force enumeration — while remaining straightforward to extend. Every design decision below is intentional and documented here so future contributors understand the reasoning, not just the implementation.

Mechanism	Implementation	Why This Approach
Password storage	bcrypt — 12 rounds	12 rounds is the 2026 standard; slow enough to resist GPU cracking, fast enough not to degrade UX
Access token	JWT — 15-minute expiry, returned in response body	Stored in client memory only (never <code>localStorage</code>); short TTL limits blast radius if intercepted
Refresh token	JWT — 7-day expiry, <code>httpOnly + Secure + SameSite=Strict</code> cookie	Cookie is inaccessible to JavaScript, eliminating XSS as an extraction vector
Token revocation	Redis allowlist keyed by <code>refresh:{userId}</code>	Logout immediately invalidates the token server-side; prevents replay after sign-out
Input validation	Zod schemas on every endpoint	Malformed data is rejected before it reaches Prisma;

Mechanism	Implementation	Why This Approach
		schema is the single source of truth
Email enumeration	Login returns identical error for wrong email or wrong password	Prevents attackers from confirming whether an email is registered
Alpha access	Invite code gate via <code>ALPHA_INVITE_ONLY</code> env flag	Toggle-controlled; switching to open registration requires only an env change

8.2 Registration Endpoint

LINUX-1-C1 Create `src/routes/auth.ts` — **Registration: POST**
`/api/auth/register` MAC

Create `apps/server/src/routes/auth.ts` and implement the full registration flow. The registration handler executes nine distinct steps in sequence; each step is numbered in the code comments below so you can trace exactly what is happening and in what order.

TRANSACTION BOUNDARY

The user record, default avatar, and personal room are created inside a single Prisma `$transaction` (Step 6). If any one of the three writes fails, none of them are committed. This guarantees that no user can exist in the database without a corresponding avatar and room.

```
import { Router, Request, Response } from 'express'; import bcrypt from 'bcryptjs'; import jwt from 'jsonwebtoken'; import crypto from 'crypto';
import { z } from 'zod'; import { prisma } from '../prisma'; import { redis } from '../redis'; import { inventoryService } from
'../services/InventoryService'; import { Resend } from 'resend'; const
router = Router(); const resend = new Resend(process.env.RESEND_API_KEY); //
— Zod validation schemas —
const registerSchema = z.object({ username: z.string().min(3,
'Username must be at least 3 characters').max(20, 'Username cannot
```

```

exceed 20 characters') .regex(/^[-a-zA-Z0-9_]+$/, 'Username may only
contain letters, numbers, and underscores'), email:
z.string().email('Invalid email address'), password:
z.string().min(8, 'Password must be at least 8
characters') .regex(/[A-Z]/, 'Password must contain at least one
uppercase letter') .regex(/[0-9]/, 'Password must contain at least one
number'), inviteCode: z.string().optional(), }); // — POST
/api/auth/register
router.post('/register', async (req: Request, res: Response) => { // 1.
Validate input const parsed = registerSchema.safeParse(req.body); if (!
parsed.success) { return res.status(400).json({ error: 'Validation
failed', fields: parsed.error.flatten().fieldErrors, }); }
const { username, email, password, inviteCode } = parsed.data; // 2. Check
invite code (only when ALPHA_INVITE_ONLY=true) let inviteRecord: any =
null; if (process.env.ALPHA_INVITE_ONLY === 'true') { if (!inviteCode)
{ return res.status(403).json({ error: 'An invite code is required
during alpha.', code: 'INVITE_REQUIRED' }); } inviteRecord = await
prisma.inviteCode.findUnique({ where: { code: inviteCode.toUpperCase() } });
if (!inviteRecord || !inviteRecord.isActive || inviteRecord.usedById || new
Date() > inviteRecord.expiresAt) { return res.status(403).json({ error:
'Invalid or expired invite code.', code: 'INVITE_INVALID' }); } //
3. Check username uniqueness (case-insensitive) const existingUsername =
await prisma.user.findFirst({ where: { username: { equals: username,
mode: 'insensitive' } }, }); if (existingUsername) { return
res.status(409).json({ error: 'Username already taken.', code:
'USERNAME_TAKEN' }); } // 4. Check email uniqueness const
existingEmail = await prisma.user.findUnique({ where: { email:
email.toLowerCase() } }); if (existingEmail) { return
res.status(409).json({ error: 'An account with this email already exists.',
code: 'EMAIL_TAKEN' }); } // 5. Hash password (12 rounds = 2026 best
practice) const passwordHash = await bcrypt.hash(password, 12); const
emailVerifyToken = crypto.randomBytes(32).toString('hex'); // 6. Create
user, avatar, personal room in a transaction const user = await prisma.
$transaction(async (tx) => { const newUser = await tx.user.create({
data: { username, email: email.toLowerCase(),
passwordHash, emailVerifyToken, }, }); // Create
default avatar await tx.avatar.create({ data: { userId:
newUser.id, bodyType: 'default', skinTone: 'light',
hairStyle: 'hair-short-01', hairColor: 'brown', eyeStyle:
'eyes-default', outfitBody: 'shirt-white', outfitLegs:
'pants-blue', outfitFeet: 'shoes-white', }, }); //
Create personal room await tx.room.create({ data: { name:
`${username}'s Room`, description: 'My personal HavenWorld space.',
ownerId: newUser.id, isPublic: false, maxOccupants: 10,
width: 10, height: 8, backgroundKey: 'map-personal-room',
theme: 'personal', }, }); return newUser; }); // 7. Grant
default free items to inventory await
inventoryService.grantDefaultItems(user.id); // 8. Mark invite code as
used if (inviteRecord) { await prisma.inviteCode.update({ where:
{ id: inviteRecord.id }, data: { usedById: user.id, usedAt: new Date(),
isActive: false }, }); } // 9. Send verification email via Resend
const verifyUrl = `${process.env.CLIENT_URL}/verify?token=${
emailVerifyToken}`; await resend.emails.send({ from:
process.env.EMAIL_FROM!, to: email, subject: 'Verify your HavenWorld
account', html: `<div style="font-family:sans-serif;max-
width:480px;margin:0 auto;padding:32px;
background:#0d0d1a;color:#fff;border-radius:16px;"> <h1
style="color:#4ecdc4;">Welcome to HavenWorld, ${username}!</h1>
<p>Click the button below to verify your email and enter the world.</p>
<a href="${verifyUrl}" style="display:inline-block;margin:24px

```

```
0;padding:14px 28px;                                background:#4ecdc4;color:#0d0d1a;text-
decoration:none;                                border-radius:8px;font-weight:bold;">
Verify My Account                                </a>                                <p style="color:#aaa;font-
size:0.85rem;">                                Link expires in 48 hours. If you didn't create this
account, ignore this email.                                </p>                                </div>                                ` ,    });    return
res.status(201).json({                                message: 'Account created! Check your email to
verify before logging in.',    });    });
```

Registration Flow — Step Summary

#	Step	Failure Response
1	Zod schema validation	400 — field-level error map
2	Invite code check (alpha only)	403 INVITE_REQUIRED / INVITE_INVALID
3	Username uniqueness (case-insensitive)	409 USERNAME_TAKEN
4	Email uniqueness	409 EMAIL_TAKEN
5	bcrypt hash (12 rounds) + token generation	500 if bcrypt fails
6	Prisma transaction: user + avatar + room	500 — all three rolled back
7	Grant default inventory items	Non-fatal; logged only
8	Mark invite code used	Non-fatal if inviteRecord is null
9	Send verification email via Resend	Non-fatal; user can request resend later

8.3 Email Verification Endpoint

LINUX-1-C2 Email Verification: GET /api/auth/verify **MAC**

Add the following handler to `src/routes/auth.ts`. The verification token is a 32-byte cryptographically random hex string generated at registration. On success, the handler sets `emailVerified: true`, nullifies the token so it cannot be reused, and redirects the client to the game login page with a `?verified=true` flag so the client can surface a success message.

```
router.get('/verify', async (req: Request, res: Response) => { const
{ token } = req.query; if (!token || typeof token !== 'string')
{ return res.status(400).json({ error: 'Verification token is missing.',
code: 'TOKEN_MISSING' }); } const user = await prisma.user.findFirst({
where: { emailVerifyToken: token }, }); if (!user) { return
res.status(400).json({ error: 'Invalid or expired verification token.', code:
'TOKEN_INVALID' }); } await prisma.user.update({ where: { id:
user.id }, data: { emailVerified: true, emailVerifyToken: null }, });
// Redirect to the game login page with a verified flag return
res.redirect(`${process.env.CLIENT_URL}?verified=true`); });
```

✓ SECURITY NOTE

Setting `emailVerifyToken: null` on verification means the same token cannot be used a second time even if an attacker intercepts the email. The token is single-use by design.

8.4 Login Endpoint

LINUX-1-C3 Login: POST /api/auth/login MAC

Add the token generation helper and cookie options above all route handlers. These are referenced by both the login and refresh endpoints. The helper is a pure function — it takes user identity fields and returns a signed access token and refresh token pair. Keeping it separate from the route handler makes it easy to unit-test in isolation.

```
function generateTokens(userId: string, username: string, role: string)
{ const accessToken = jwt.sign( { userId, username, role },
process.env.JWT_ACCESS_SECRET!, { expiresIn:
process.env.JWT_ACCESS_EXPIRES ?? '15m' } ); const refreshToken =
jwt.sign( { userId }, process.env.JWT_REFRESH_SECRET!,
{ expiresIn: process.env.JWT_REFRESH_EXPIRES ?? '7d' } ); return
{ accessToken, refreshToken }; } const REFRESH_COOKIE_OPTIONS =
{ httpOnly: true, secure: process.env.NODE_ENV === 'production',
sameSite: 'strict' as const, maxAge: 7 * 24 * 60 * 60 * 1000, path:
'/api/auth', };
```

Now add the login route. The handler deliberately returns the same error message for a missing account and a wrong password — this prevents attackers from using the login form to enumerate which email addresses are registered in the system.

```
const loginSchema = z.object({ email: z.string().email(), password:
z.string().min(1), }); router.post('/login', async (req: Request, res:
Response) => { const parsed = loginSchema.safeParse(req.body); if (!
parsed.success) { return res.status(400).json({ error: 'Invalid
request.', code: 'VALIDATION_ERROR' }); } const { email, password } =
parsed.data; // Find by email – never reveal which field was wrong
(security best practice) const user = await
prisma.user.findUnique({ where: { email: email.toLowerCase() }, });
if (!user) { return res.status(401).json({ error: 'Invalid email or
password.', code: 'INVALID_CREDENTIALS' }); } if (!user.emailVerified) {
return res.status(403).json({ error: 'Please verify your email address
before logging in.', code: 'EMAIL_NOT_VERIFIED', }); } if
(user.status === 'BANNED') { return res.status(403).json({ error: 'This
account has been suspended.', code: 'ACCOUNT_BANNED' }); } const
passwordValid = await bcrypt.compare(password, user.passwordHash); if (!
passwordValid) { return res.status(401).json({ error: 'Invalid email or
password.', code: 'INVALID_CREDENTIALS' }); } const { accessToken,
refreshToken } = generateTokens(user.id, user.username, user.role); //
Store refresh token in Redis (7-day TTL) await redis.setEx(`refresh:$
{user.id}`, 7 * 24 * 3600, refreshToken); // Update last login timestamp
await prisma.user.update({ where: { id: user.id }, data:
{ lastLoginAt: new Date() }, }); // Set httpOnly refresh token cookie
res.cookie('refreshToken', refreshToken, REFRESH_COOKIE_OPTIONS); return
res.json({ accessToken, user: { id: user.id, username: user.username,
role: user.role }, }); });
```

Login — Guard Rail Summary

Guard	HTTP Status	Code
Zod validation fails	400	VALIDATION_ERROR
Email not found	401	INVALID_CREDENTIALS
Email not verified	403	EMAIL_NOT_VERIFIED
Account is banned	403	ACCOUNT_BANNED
Password mismatch	401	INVALID_CREDENTIALS
Success	200	accessToken + user object in body; refreshToken cookie set

8.5 Token Refresh, Logout, and Continuing to Export

LINUX-1-C4 Token Refresh: POST /api/auth/refresh MAC

The refresh endpoint is what keeps a logged-in user's session alive without requiring them to re-enter their password. It reads the `httpOnly` cookie, verifies the JWT

signature, and then cross-checks it against the Redis allowlist. If an attacker steals the cookie and calls this endpoint after the user has logged out, Redis returns a non-matching value and the request is rejected. The endpoint also performs refresh token rotation — a new refresh token is issued on every successful call, invalidating the previous one.

```
router.post('/refresh', async (req: Request, res: Response) => {  const
{ refreshToken } = req.cookies;  if (!refreshToken) {    return
res.status(401).json({ error: 'No refresh token.', code:
'NO_REFRESH_TOKEN' });  }  let payload: any;  try {    payload =
jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET!);  } catch
{    return res.status(401).json({ error: 'Refresh token invalid or
expired.', code: 'REFRESH_INVALID' });  }  // Check Redis allowlist –
prevents reuse after logout  const stored = await redis.get(`refresh:${
payload.userId}`);  if (stored !== refreshToken) {    return
res.status(401).json({ error: 'Refresh token has been revoked.', code:
'REFRESH_REVOKED' });  }  // Load current user (in case role changed since
last login)  const user = await prisma.user.findUnique({    where: { id:
payload.userId },    select: { id: true, username: true, role: true, status:
true },  });  if (!user || user.status === 'BANNED') {    return
res.status(403).json({ error: 'Account not found or suspended.', code:
'ACCOUNT_INVALID' });  }  // Issue new access token + rotate refresh token
const { accessToken, refreshToken: newRefreshToken } =
generateTokens(user.id, user.username, user.role);  await
redis.setEx(`refresh:${user.id}`, 7 * 24 * 3600, newRefreshToken);
res.cookie('refreshToken', newRefreshToken, REFRESH_COOKIE_OPTIONS);
return res.json({ accessToken }); });
```

LINUX-1-C5 Logout: POST /api/auth/logout MAC

Logout immediately deletes the Redis allowlist entry for this user's refresh token. Even if someone captures the cookie value from a network trace before logout, attempting to use it after this point will fail the allowlist check. The cookie is also cleared client-side via `res.clearCookie`. After the logout handler, export the router.

```
router.post('/logout', async (req: Request, res: Response) => {  const
{ refreshToken } = req.cookies;  if (refreshToken) {    try {      const
payload = jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET!) as any;
// Invalidate the refresh token in Redis immediately      await
redis.del(`refresh:${payload.userId}`);    } catch {      // Token already
expired – no action needed    }  }  res.clearCookie('refreshToken',
{ ...REFRESH_COOKIE_OPTIONS, maxAge: 0 });  return res.json({ message:
'Logged out successfully.' }); });  export default router;
```

8.6 Users, Rooms, Friends, and Reports Routes

This file exposes the current user's profile, avatar configuration, inventory, and personal room ID, plus a public profile lookup by username. All routes are protected by the `requireAuth` middleware that validates the JWT access token from the Authorization header. The `select` clauses deliberately exclude `passwordHash` and `emailVerifyToken` from every response.

```
import { Router } from 'express'; import { requireAuth, AuthRequest } from
'../middleware/auth'; import { prisma } from '../prisma'; import
{ inventoryService } from '../services/InventoryService'; const router =
Router(); // GET /api/users/me – current user profile router.get('/me',
requireAuth, async (req: AuthRequest, res) => { const user = await
prisma.user.findUnique({ where: { id: req.user!.userId }, select:
{ id: true, username: true, email: true, role: true, status:
true, createdAt: true, lastLoginAt: true }, }); if (!user) return
res.status(404).json({ error: 'User not found.' }); return
res.json(user); }); // GET /api/users/me/avatar – current user avatar config
router.get('/me/avatar', requireAuth, async (req: AuthRequest, res) =>
{ const avatar = await prisma.avatar.findUnique({ where: { userId:
req.user!.userId } }); if (!avatar) return res.status(404).json({ error:
'Avatar not found.' }); return res.json(avatar); }); // GET
/api/users/me/inventory – items in user's inventory with full item details
router.get('/me/inventory', requireAuth, async (req: AuthRequest, res) => {
const inventory = await inventoryService.getUserInventory(req.user!.userId);
return res.json(inventory); }); // GET /api/users/me/room – user's personal
room ID router.get('/me/room', requireAuth, async (req: AuthRequest, res) =>
{ const room = await prisma.room.findFirst({ where: { ownerId:
req.user!.userId }, select: { id: true, name: true, width: true, height:
true }, }); if (!room) return res.status(404).json({ error: 'Personal
room not found.' }); return res.json(room); }); // GET
/api/users/:username – public profile of another user
router.get('/:username', requireAuth, async (req: AuthRequest, res) =>
{ const user = await prisma.user.findFirst({ where: { username:
{ equals: req.params.username, mode: 'insensitive' } }, select: { id:
true, username: true, createdAt: true }, }); if (!user) return
res.status(404).json({ error: 'User not found.' }); return
res.json(user); }); export default router;
```

Users Route Map

Method	Path	Auth	Returns
GET	/api/users/me	Required	Full profile (no password fields)
GET	/api/users/me/avatar	Required	Avatar configuration object
GET	/api/users/me/inventory	Required	Array of inventory items with details

Method	Path	Auth	Returns
GET	/api/users/me/room	Required	Personal room id, name, dimensions
GET	/api/users/:username	Required	Public profile (id, username, createdAt)

LINUX-1-C7 Create src/routes/rooms.ts MAC

The rooms route handles listing all public rooms (with live occupant counts injected from the RoomManager singleton), fetching a single room including its furniture, and placing or removing furniture in a personal room. The furniture placement endpoint verifies that the item being placed actually exists in the requesting user's inventory before writing to the database — this is the server-side ownership gate that prevents clients from placing items they do not own.

```
import { Router } from 'express'; import { requireAuth, AuthRequest } from
'../middleware/auth'; import { prisma } from '../prisma'; import
{ roomManager } from '../services/RoomManager'; const router = Router(); //
GET /api/rooms – list all public rooms with live occupant counts
router.get('/', requireAuth, async (_req, res) => { const rooms = await
prisma.room.findMany({ where: { isPublic: true }, select: { id: true,
name: true, description: true, maxOccupants: true, theme: true },
orderBy: { name: 'asc' }, }); // Inject live player count from
RoomManager in-memory state const withOccupants = rooms.map((room) => ({
...room, occupants: roomManager.getOccupantCount(room.id), }));
return res.json(withOccupants); }); // GET /api/rooms/:id – single room
details including furniture router.get('/:id', requireAuth, async (req:
AuthRequest, res) => { const room = await
prisma.room.findUnique({ where: { id: req.params.id }, include: {
furniture: { include: { item: true }, orderBy: { layer:
'asc' } }, }, }); if (!room) return res.status(404).json({ error:
'Room not found.' }); // Private rooms can only be seen by their owner
if (!room.isPublic && room.ownerId !== req.user!.userId) { return
res.status(403).json({ error: 'This room is private.', code:
'ROOM_PRIVATE' }); } return res.json(room); }); // POST
/api/rooms/:id/furniture – place furniture in personal room
router.post('/:id/furniture', requireAuth, async (req: AuthRequest, res) => {
const room = await prisma.room.findUnique({ where: { id: req.params.id } });
if (!room || room.ownerId !== req.user!.userId) { return
res.status(403).json({ error: 'You do not own this room.', code:
'NOT_ROOM_OWNER' }); } const { itemId, x, y, z = 0, rotation = 0, layer =
0 } = req.body; // Verify item is in the user's inventory const owned =
await prisma.inventory.findUnique({ where: { userId_itemId: { userId:
req.user!.userId, itemId } }, }); if (!owned) return
res.status(403).json({ error: 'You do not own this item.', code:
'ITEM_NOT_OWNED' }); const placed = await prisma.roomFurniture.create({
```

```

data: { roomId: req.params.id, itemId, placedBy: req.user!.userId, x, y, z,
rotation, layer }, include: { item: true }, }); // Update
RoomManager cached furniture list const item = placed.item;
roomManager.addFurniture(req.params.id, { id: placed.id, itemId,
spriteKey: item.spriteKey, x, y, z, rotation, layer, }); return
res.status(201).json(placed); }); // DELETE
/api/rooms/:id/furniture/:furnitureId – remove furniture
router.delete('/:id/furniture/:furnitureId', requireAuth, async (req:
AuthRequest, res) => { const room = await prisma.room.findUnique({ where: {
id: req.params.id } }); if (!room || room.ownerId !== req.user!.userId) {
return res.status(403).json({ error: 'You do not own this room.', code:
'NOT_ROOM_OWNER' }); } await prisma.roomFurniture.delete({ where: { id:
req.params.furnitureId } }); roomManager.removeFurniture(req.params.id,
req.params.furnitureId); return res.json({ message: 'Furniture
removed.' }); }); export default router;

```

LINUX-1-C8 Create src/routes/friends.ts MAC

The friends system is built on the bidirectional `Friend` model from the Prisma schema. A friendship record can be created by either party as a `PENDING` request; it becomes `ACCEPTED` when the addressee responds. The online status of each friend is resolved by checking a Redis set (`online_users`) — the same set that the Socket.io disconnect handler manages. This means online status is always consistent with who actually has an open WebSocket connection.

```

import { Router } from 'express'; import { requireAuth, AuthRequest } from
'../middleware/auth'; import { prisma } from '../prisma'; import { redis }
from '../redis'; const router = Router(); // GET /api/friends – list
accepted friends with online status router.get('/', requireAuth, async (req:
AuthRequest, res) => { const userId = req.user!.userId; const friends =
await prisma.friend.findMany({ where: { OR: [{ requesterId:
userId }, { addresseeId: userId }], status: 'ACCEPTED', },
include: { requester: { select: { id: true, username: true } },
addressee: { select: { id: true, username: true } }, }, }); const
result = await Promise.all(friends.map(async (f) => { const friend =
f.requesterId === userId ? f.addressee : f.requester; const isOnline =
await redis.sIsMember('online_users', friend.id); return { ...friend,
isOnline }; })); return res.json(result); }); // GET
/api/friends/requests – pending incoming requests router.get('/requests',
requireAuth, async (req: AuthRequest, res) => { const requests = await
prisma.friend.findMany({ where: { addresseeId: req.user!.userId, status:
'PENDING' }, include: { requester: { select: { id: true, username: true } }
}, }); return res.json(requests); }); // POST /api/friends/request –
send a friend request router.post('/request', requireAuth, async (req:
AuthRequest, res) => { const { targetUserId } = req.body; if
(targetUserId === req.user!.userId) { return
res.status(400).json({ error: 'You cannot send a friend request to yourself.'
}); } const existing = await prisma.friend.findFirst({ where: {
OR: [ { requesterId: req.user!.userId, addresseeId: targetUserId },
{ requesterId: targetUserId, addresseeId: req.user!.userId }, ], },
}); if (existing) return res.status(409).json({ error: 'Friend request

```

```

already exists.' });    const request = await
prisma.friend.create({    data: { requesterId: req.user!.userId,
addresseeId: targetUserId },    });    return
res.status(201).json(request); }); // POST /api/friends/accept/:requesterId
– accept a friend request router.post('/accept/:requesterId', requireAuth,
async (req: AuthRequest, res) => {    const updated = await
prisma.friend.updateMany({    where: { requesterId: req.params.requesterId,
addresseeId: req.user!.userId, status: 'PENDING' },    data: { status:
'ACCEPTED' },    });    if (updated.count === 0) return
res.status(404).json({ error: 'Friend request not found.' });    return
res.json({ message: 'Friend request accepted.' }); }); // DELETE
/api/friends/:friendId – remove a friend or decline a request
router.delete('/:friendId', requireAuth, async (req: AuthRequest, res) => {
await prisma.friend.deleteMany({    where: {    id: req.params.friendId,
OR: [{ requesterId: req.user!.userId }, { addresseeId: req.user!.userId }],
},    });    return res.json({ message: 'Friend removed.' }); }); export
default router;

```

LINUX-1-C9 Create src/routes/reports.ts MAC

The player report endpoint accepts structured abuse reports from authenticated users. The reason field is an enum — free-form strings are not accepted, preventing garbage data from polluting the moderation queue. The description field is capped at 500 characters. Reports are stored in the database and surfaced in the admin moderation panel covered in Part 4C.

```

import { Router } from 'express'; import { requireAuth, AuthRequest } from
'../middleware/auth'; import { prisma } from '../prisma'; import { z } from
'zod'; const router = Router(); const reportSchema =
z.object({    reportedUserId: z.string().uuid(),    reason:
z.enum(['HARASSMENT', 'SPAM', 'INAPPROPRIATE_CONTENT',
'IMPERSONATION', 'CHEATING', 'OTHER']),    description:
z.string().max(500).default(''), }); // POST /api/reports – submit a player
report router.post('/', requireAuth, async (req: AuthRequest, res) =>
{    const parsed = reportSchema.safeParse(req.body);    if (!parsed.success) {
return res.status(400).json({    error: 'Invalid report data.',
fields: parsed.error.flatten().fieldErrors,    });    }    if
(parsed.data.reportedUserId === req.user!.userId) {    return
res.status(400).json({ error: 'You cannot report yourself.' });    }    const
report = await prisma.report.create({    data: { reporterId:
req.user!.userId, ...parsed.data },    });    return
res.status(201).json({ reportId: report.id, message: 'Report submitted. Thank
you.' }); }); export default router;

```

⚠ REGISTER ALL ROUTES IN src/index.ts

After completing C6–C9, import and mount all four route files in `src/index.ts`:

```
app.use('/api/auth', authRouter);
app.use('/api/users', usersRouter);
app.use('/api/rooms', roomsRouter);
app.use('/api/friends', friendsRouter);
app.use('/api/reports', reportsRouter);
```

Failure to register the routers means the endpoints will return 404 silently.

Section 9: RoomManager Service MAC

9.1 In-Memory Game State Design

The RoomManager is the nerve center of all real-time game state. It holds, in RAM, the authoritative answer to three questions: who is in which room, where exactly each player is standing, and what furniture and chat history exist in each room right now. It is intentionally *not* a database-backed service — the database is the source of truth for permanent data; RoomManager is the source of truth for ephemeral, real-time transient state that changes dozens of times per second and cannot be persisted at that frequency without crippling the database.

The singleton pattern is critical here. Because Node.js loads modules once and caches them, `export const roomManager = new RoomManager()` at the bottom of the file guarantees that every import of this module — whether from the Socket.io handler, the rooms route, or any future service — receives the exact same object instance. There is no shared state confusion across threads because Node.js is single-threaded; all socket events and HTTP requests are handled on the same event loop.

LINUX-1-D1 Create `src/services/RoomManager.ts` MAC

```
import type { PlayerState, ChatMessage, FurnitureState } from
'@havenworld/shared'; interface RoomState {  players: Map<string,
PlayerState>;  // socketId → PlayerState  chatHistory: ChatMessage[];
```

```

// last 50 messages (ring buffer)   furniture: FurnitureState[];           //
loaded from DB on first join   loaded: boolean;           //
whether furniture was loaded from DB }   class RoomManager {   private rooms =
new Map<string, RoomState>();   private socketToRoom = new Map<string,
string>();   // socketId → roomId   private socketToUser = new Map<string,
string>();   // socketId → userId   // — Room lifecycle

private
ensureRoom(roomId: string): RoomState {   if (!this.rooms.has(roomId)) {
this.rooms.set(roomId, {   players: new Map(),   chatHistory: [],
furniture: [],   loaded: false,   });   }   return
this.rooms.get(roomId)!;   }   // — Player join / leave

joinRoom(roomId:
string, socketId: string, player: PlayerState): void {   const room =
this.ensureRoom(roomId);   room.players.set(socketId, player);
this.socketToRoom.set(socketId, roomId);   this.socketToUser.set(socketId,
player.id);   }   leaveRoom(socketId: string): { roomId: string; playerId:
string } | null {   const roomId = this.socketToRoom.get(socketId);   if
(!roomId) return null;   const room = this.rooms.get(roomId);   const
player = room?.players.get(socketId);   room?.players.delete(socketId);
this.socketToRoom.delete(socketId);   this.socketToUser.delete(socketId);
// Public rooms are kept alive when empty (saves re-loading furniture)   //
Personal rooms are cleaned up when empty to free memory   if (room &&
room.players.size === 0 && roomId.startsWith('personal-'))
{   this.rooms.delete(roomId);   }   return player ? { roomId,
playerId: player.id } : null;   }   // — Player movement

movePlayer(socketId: string, x: number, y: number, direction: string,
isMoving: boolean): void {   const roomId =
this.socketToRoom.get(socketId);   if (!roomId) return;   const player =
this.rooms.get(roomId)?.players.get(socketId);   if (player)
{   player.x = x;   player.y = y;   player.direction = direction
as PlayerState['direction'];   player.isMoving =
isMoving;   }   }   // — Chat history

addChatMessage(roomId: string, message: ChatMessage): void {   const room =
this.ensureRoom(roomId);   room.chatHistory.push(message);   if
(room.chatHistory.length > 50) {   room.chatHistory.shift();   // keep
last 50 – ring buffer   }   }   // — Furniture

setFurniture(roomId: string, furniture: FurnitureState[]): void {   const
room = this.ensureRoom(roomId);   room.furniture = furniture;
room.loaded = true;   }   addFurniture(roomId: string, item:
FurnitureState): void {   this.ensureRoom(roomId).furniture.push(item);   }
removeFurniture(roomId: string, furnitureId: string): void {   const room =
this.rooms.get(roomId);   if (room) {   room.furniture =
room.furniture.filter(f => f.id !== furnitureId);   }   }
isFurnitureLoaded(roomId: string): boolean {   return
this.rooms.get(roomId)?.loaded ?? false;   }   // — Queries

getRoomState(roomId: string): RoomState | undefined {   return
this.rooms.get(roomId);   }   getPlayerRoom(socketId: string): string |
undefined {   return this.socketToRoom.get(socketId);   }
getUserId(socketId: string): string | undefined {   return
this.socketToUser.get(socketId);   }   getOccupantCount(roomId: string):
number {   return this.rooms.get(roomId)?.players.size ?? 0;   }
getAllRoomOccupants(): Map<string, number> {   const result = new
Map<string, number>();   this.rooms.forEach((state, roomId) =>
{   result.set(roomId, state.players.size);   });   return
result;   } } // Singleton – shared across the entire server process export
const roomManager = new RoomManager();

```

RoomManager Internal Data Structure

Map	Key	Value	Purpose
<code>rooms</code>	<code>roomId</code>	RoomState object	All live state per room
<code>socketToRoom</code>	<code>socketId</code>	<code>roomId</code>	Fast lookup: which room is this socket in?
<code>socketToUser</code>	<code>socketId</code>	<code>userId</code>	Fast lookup: which user owns this socket?
<code>RoomState.players</code>	<code>socketId</code>	PlayerState	Live position + avatar for every player in the room
<code>RoomState.chatHistory</code>	—	ChatMessage[]	Ring buffer, max 50 messages
<code>RoomState.furniture</code>	—	FurnitureState[]	Cached from DB on first join; updated live after that

Section 10: Moderation Service MAC

LINUX-1-E1 Create `src/services/ModerationService.ts` MAC

The ModerationService is a pure utility module — it has no class, no state of its own beyond the in-process rate limit store, and no database dependency. It exports three functions: `moderateMessage` for profanity filtering, `checkRateLimit` for per-socket throttling, and `clearRateLimitEntry` for cleanup on disconnect. The profanity filter uses the `bad-words` npm package with the replacement character set to `*` — filtered messages are still stored in the database with the `isFiltered: true` flag set, so moderators can see what was said while players in the room only see the sanitized version.

The rate limiter is keyed by `socketId`, not by IP address. This is intentional: sockets are already authenticated, so rate limiting per socket is equivalent to rate limiting per user session. IP-based rate limiting would fail in shared NAT scenarios (e.g., a school or office network) where many legitimate users share one IP.

```
import Filter from 'bad-words'; const filter = new Filter({ placeholder: '*'
}); // Add any HavenWorld-specific words to block: //
filter.addWords('word1', 'word2'); export interface ModerationResult
{ filtered: string; wasFiltered: boolean; severity: 'none' | 'mild' |
'severe'; } export function moderateMessage(content: string):
ModerationResult { const trimmed = content.trim(); try { const
filtered = filter.clean(trimmed); const wasFiltered = filtered !==
trimmed; return { filtered, wasFiltered, severity:
wasFiltered ? 'mild' : 'none', }; } catch { // bad-words throws
when the ENTIRE string is considered profane return { filtered: '***',
wasFiltered: true, severity: 'severe' }; } } // Per-socket rate limiter
(not per IP – sockets are already authenticated) const rateLimitStore = new
Map<string, { count: number; resetAt: number }>(); export function
checkRateLimit( socketId: string, maxMessages: number = 5, windowMs:
number = 3000 ): boolean { const now = Date.now(); const record =
rateLimitStore.get(socketId) ?? { count: 0, resetAt: now + windowMs }; if
(now > record.resetAt) { record.count = 0; record.resetAt = now +
windowMs; } record.count += 1; rateLimitStore.set(socketId, record);
return record.count <= maxMessages; } export function
clearRateLimitEntry(socketId: string): void
{ rateLimitStore.delete(socketId); }
```

ModerationService — Function Reference

Function	Parameters	Returns	Called From
<code>moderateMessage</code>	<code>content: string</code>	<code>ModerationResult</code> (filtered text, flag, severity)	<code>chat:send</code> handler
<code>checkRateLimit</code>	<code>socketId</code> , <code>maxMessages</code> , <code>windowMs</code>	<code>boolean</code> — true if within limit	<code>chat:send</code> handler
<code>clearRateLimitEntry</code>	<code>socketId: string</code>	<code>void</code>	<code>disconnect</code> handler

Section 11: Socket.io Real-Time Handlers MAC

This is the most complex single file in the HavenWorld server. It orchestrates everything built in Sections 8 through 10 into a live, bidirectional event pipeline. Read every comment carefully — the ordering of operations within each handler is

not arbitrary. A handler that updates the database before validating ownership, or that broadcasts before updating local state, will introduce subtle bugs that are extremely difficult to reproduce and debug.

11.1 Socket Auth Middleware

LINUX-1-F1 Create `src/sockets/index.ts` — Auth Middleware **MAC**

The socket authentication middleware runs on every new WebSocket connection attempt, before any `socket.on()` handler fires. It reads the JWT access token from `socket.handshake.auth.token` (the client sends this during the Socket.io handshake), verifies it, and attaches the decoded payload to `socket.data.user`. If verification fails, the connection is rejected with an error before it is accepted. The move rate limiter is defined at module scope — it is separate from the chat rate limiter because movement events are much higher frequency (up to 20 per second) and require a different limit.

```
import { Server, Socket } from 'socket.io'; import jwt from 'jsonwebtoken';
import { prisma } from '../prisma'; import { redis } from '../redis'; import
{ roomManager } from '../services/RoomManager'; import { moderateMessage,
checkRateLimit, clearRateLimitEntry } from '../services/ModerationService';
import { SOCKET_EVENTS } from '@havenworld/shared'; import type
{ PlayerState, FurnitureState } from '@havenworld/shared'; // — Socket
authentication middleware // Every
socket connection must provide a valid JWT access token. // This runs before
any socket.on() handler. function socketAuthMiddleware(socket: Socket, next:
(err?: Error) => void): void { const token = socket.handshake.auth?.token;
if (!token || typeof token !== 'string') { return next(new
Error('AUTH_REQUIRED')); } try { const payload = jwt.verify(token,
process.env.JWT_ACCESS_SECRET!) as { userId: string; username:
string; role: string; }; socket.data.user = payload;
next(); } catch { next(new Error('AUTH_INVALID')); } } // Move rate
limiter (separate from chat — higher frequency) const moveRateLimiter = new
Map<string, { count: number; resetAt: number }>(); function
checkMoveRateLimit(socketId: string): boolean { const now = Date.now();
const rec = moveRateLimiter.get(socketId) ?? { count: 0, resetAt: now +
1000 }; if (now > rec.resetAt) { rec.count = 0; rec.resetAt = now + 1000; }
rec.count += 1; moveRateLimiter.set(socketId, rec); return rec.count <=
20; // max 20 position updates per second } export function
registerSocketHandlers(io: Server): void { io.use(socketAuthMiddleware);
io.on('connection', (socket: Socket) => { const { userId, username,
role } = socket.data.user as { userId: string; username: string; role:
string; }; console.log(`[Socket] Connected: ${username} (${
socket.id})`);
```

11.2 auth:join Handler

LINUX-1-F2 Implement auth:join in src/sockets/index.ts MAC

The `auth:join` event is the first event the client emits after the WebSocket connection is established. It provides the `roomId` the player wants to enter. The handler performs a complete onboarding sequence: ban check, avatar load, furniture load (from DB if not already cached by RoomManager), spawn position calculation, Socket.io room join, RoomManager registration, Redis online-status update, full room state delivery to the joining player, announcement to existing players, and friend online notification. Each of these must complete before the client is considered fully in the room.

```
// — auth:join —
// First event the client emits after connecting. // Loads avatar +
// furniture from DB, joins the room, sends room:state.
socket.on(SOCKET_EVENTS.AUTH_JOIN, async ({ roomId }: { roomId: string }) =>
{
  try { // Check ban status const dbUser = await
prisma.user.findUnique({ where: { id: userId }, select: {
status: true, mutedUntil: true }, }); if (dbUser?.status ===
'BANNED') { socket.emit(SOCKET_EVENTS.AUTH_ERROR, { code: 'BANNED',
message: 'Your account has been suspended.' });
socket.disconnect(true); return; } // Load avatar
from DB const avatar = await prisma.avatar.findUnique({ where:
{ userId } }); if (!avatar)
{ socket.emit(SOCKET_EVENTS.AUTH_ERROR, { code: 'NO_AVATAR',
message: 'Avatar not found.' }); return; } const
avatarData = { bodyType: avatar.bodyType, skinTone:
avatar.skinTone, hairStyle: avatar.hairStyle, hairColor:
avatar.hairColor, eyeStyle: avatar.eyeStyle, eyeColor:
avatar.eyeColor, outfitHead: avatar.outfitHead,
outfitFace: avatar.outfitFace, outfitBody: avatar.outfitBody,
outfitLegs: avatar.outfitLegs, outfitFeet: avatar.outfitFeet,
outfitBack: avatar.outfitBack, outfitHand: avatar.outfitHand,
}; // Load furniture from DB if not already cached in RoomManager
if (!roomManager.isFurnitureLoaded(roomId)) { const dbFurniture =
await prisma.roomFurniture.findMany({ where: { roomId },
include: { item: true }, orderBy: { layer: 'asc' }, });
const furniture: FurnitureState[] = dbFurniture.map((f) => ({ id:
f.id, itemId: f.itemId, spriteKey: f.item.spriteKey,
x: f.x, y: f.y, z: f.z, rotation: f.rotation, layer:
f.layer, })); roomManager.setFurniture(roomId,
furniture); } // Build spawn position — center of the room
const dbRoom = await prisma.room.findUnique({ where: { id:
roomId }, select: { width: true, height: true, name: true },
}); const spawnX = Math.floor((dbRoom?.width ?? 20) / 2) * 32;
const spawnY = Math.floor((dbRoom?.height ?? 15) / 2) * 32; const
player: PlayerState = { id: userId, username,
avatar: avatarData, x: spawnX, y: spawnY,
```

```

direction: 'down',          isMoving: false,          };          // Join the
Socket.io room and register in RoomManager          socket.join(roomId);
roomManager.joinRoom(roomId, socket.id, player);          // Track online
status in Redis          await redis.sAdd('online_users', userId);          //
Send full room state to the joining player only          const roomState =
roomManager.getRoomState(roomId)!;
socket.emit(SOCKET_EVENTS.ROOM_STATE, {          roomId,          roomName:
dbRoom?.name ?? roomId,          players:
Array.from(roomState.players.values()),          furniture:
roomState.furniture,          chatHistory:
roomState.chatHistory,          });          // Notify all OTHER players in
the room that someone new joined
socket.to(roomId).emit(SOCKET_EVENTS.ROOM_PLAYER_JOINED, player);          //
Notify online friends          const friendships = await
prisma.friend.findMany({          where: {          OR: [{ requesterId:
userId }, { addresseeId: userId }],          status: 'ACCEPTED',
},          });          const friendIds = friendships.map((f) =>
f.requesterId === userId ? f.addresseeId : f.requesterId          );
if (friendIds.length > 0) {          const allSockets = await
io.fetchSockets();          for (const friendId of friendIds)
{          const friendSocket = allSockets.find((s) => s.data.user?.userId
=== friendId);          friendSocket?.emit(SOCKET_EVENTS.FRIEND_ONLINE,
{ userId, username });          }          } catch (err)
{          console.error('[Socket] auth:join error:', err);
socket.emit(SOCKET_EVENTS.AUTH_ERROR, { code: 'SERVER_ERROR', message:
'Failed to join room.' });          }          });

```

auth:join — Operation Sequence

#	Operation	Failure Behavior
1	DB: check ban status	Emit AUTH_ERROR, force disconnect
2	DB: load avatar record	Emit AUTH_ERROR, return early
3	RoomManager: check furniture cache	—
4	DB: load furniture if not cached	Non-fatal; room will have no furniture
5	DB: load room dimensions for spawn point	Falls back to 20×15 defaults
6	socket.join(roomId) + RoomManager.joinRoom()	—
7	Redis: sAdd to online_users	Non-fatal; online status will be stale
8	Emit ROOM_STATE to joining socket	—
9	Broadcast ROOM_PLAYER_JOINED to room	—

#	Operation	Failure Behavior
10	Notify online friends via FRIEND_ONLINE	Non-fatal; silently skipped

11.3 player:move, chat:send, avatar:update, and disconnect Handlers

LINUX-1-F3 Implement remaining handlers in `src/sockets/index.ts` **MAC**

These four handlers complete the Socket.io surface. They continue inside the same `io.on('connection')` callback, after the `auth:join` handler above. The `player:move` handler is the highest-frequency handler in the system and must be as lightweight as possible — it does no database I/O, no async awaits, and no external service calls. It performs three synchronous checks (rate limit, room membership, coordinate bounds), updates the in-memory map, and broadcasts. Total per-event overhead is under 1ms.

```
// — player:move —
// Client sends their new position after each tile step.
socket.on(SOCKET_EVENTS.PLAYER_MOVE, (data: { x: number; y: number;
direction: string; roomId: string; isMoving: boolean }) => { //
Rate limit: max 20 position updates per second if (!
checkMoveRateLimit(socket.id)) return; // Verify socket is actually in
the claimed room const currentRoom =
roomManager.getPlayerRoom(socket.id); if (currentRoom !== data.roomId)
return; // Basic coordinate sanity check (64px tile × 100 tiles =
6400px max) if (data.x < 0 || data.y < 0 || data.x > 9600 || data.y >
9600) return; // Update in-memory state
roomManager.movePlayer(socket.id, data.x, data.y, data.direction,
data.isMoving); // Broadcast to everyone else in the room (not the
sender) socket.to(data.roomId).emit(SOCKET_EVENTS.PLAYER_POSITION, {
playerId: userId, x: data.x, y: data.y, direction:
data.direction, isMoving: data.isMoving, }); // —
chat:send
socket.on(SOCKET_EVENTS.CHAT_SEND, async (data: { content: string; roomId?:
string }) => { // Rate limit: 5 messages per 3 seconds if (!
checkRateLimit(socket.id, 5, 3000))
{ socket.emit(SOCKET_EVENTS.CHAT_ERROR, { code: 'RATE_LIMITED',
message: 'Please slow down!' }); return; } const content
= data.content?.trim(); if (!content || content.length === 0 ||
content.length > 200) return; // Check mute status (re-query DB for
up-to-date status) const dbUser = await prisma.user.findUnique({
where: { id: userId }, select: { status: true, mutedUntil: true },
}); if (dbUser?.status === 'MUTED') { const stillMuted = !
dbUser.mutedUntil || new Date() < dbUser.mutedUntil; if (stillMuted)
{ socket.emit(SOCKET_EVENTS.CHAT_ERROR, { code: 'MUTED', message:
```

```

'You are currently muted.' }); return; } // Auto-
unmute if the mute period has expired await
prisma.user.update({ where: { id: userId }, data: { status: 'ACTIVE',
mutedUntil: null } }); } // Determine the room (use RoomManager
if client doesn't send roomId) const roomId = data.roomId ??
roomManager.getPlayerRoom(socket.id); if (!roomId) return; //
Run through profanity filter const { filtered, wasFiltered } =
moderateMessage(content); // Persist to database for moderation audit
log const saved = await prisma.chatMessage.create({ data:
{ roomId, senderId: userId, content: filtered, isFiltered: wasFiltered },
}); const message = { id: saved.id, senderId: userId,
senderName: username, content: filtered, isFiltered:
wasFiltered, timestamp: saved.createdAt.toISOString(), };
// Add to in-memory chat history (keeps last 50)
roomManager.addChatMessage(roomId, message); // Broadcast to ALL
players in the room INCLUDING the sender // (sender needs server
confirmation that their message was delivered)
io.to(roomId).emit(SOCKET_EVENTS.CHAT_MESSAGE, message); }); // —
avatar:update
socket.on(SOCKET_EVENTS.AVATAR_UPDATE, async (avatarData: Record<string,
string | null>) => { try { // Collect all non-null item IDs
from the update const itemFields =
['outfitHead', 'outfitFace', 'outfitBody',
'outfitLegs', 'outfitFeet', 'outfitBack', 'outfitHand']; const
requestedItemIds = itemFields .map((f) =>
avatarData[f]) .filter((v): v is string => typeof v === 'string');
// Verify all requested items are in the user's inventory if
(requestedItemIds.length > 0) { const count = await
prisma.inventory.count({ where: { userId, itemId: { in:
requestedItemIds } }, }); if (count !==
requestedItemIds.length) { socket.emit(SOCKET_EVENTS.AUTH_ERROR,
{ code: 'INVALID_ITEMS', message: 'One or more
items are not in your inventory.', }); return;
} } // Save new avatar config to database await
prisma.avatar.update({ where: { userId }, data: {
skinTone: avatarData.skinTone ?? undefined, hairstyle:
avatarData.hairStyle ?? undefined, hairColor:
avatarData.hairColor ?? undefined, eyeStyle:
avatarData.eyeStyle ?? undefined, outfitHead:
avatarData.outfitHead, outfitFace: avatarData.outfitFace,
outfitBody: avatarData.outfitBody, outfitLegs:
avatarData.outfitLegs, outfitFeet: avatarData.outfitFeet,
outfitBack: avatarData.outfitBack, outfitHand:
avatarData.outfitHand, }, }); // Update in-memory
player state so other players see the change immediately const roomId
= roomManager.getPlayerRoom(socket.id); if (roomId) { const
playerState = roomManager.getRoomState(roomId)?.players.get(socket.id);
if (playerState) { Object.assign(playerState.avatar, avatarData);
} // Broadcast to others so their client re-renders this avatar
socket.to(roomId).emit(SOCKET_EVENTS.AVATAR_CHANGED, { playerId:
userId, avatar: avatarData, }); } } catch
(err) { console.error('[Socket] avatar:update error:', err); }
}); // — disconnect

socket.on('disconnect', async (reason) => { console.log(`[Socket]
Disconnected: ${username} (${reason})`); // Remove from room and get
which room they were in const left = roomManager.leaveRoom(socket.id);
if (left) { // Tell everyone in the room this player left
io.to(left.roomId).emit(SOCKET_EVENTS.ROOM_PLAYER_LEFT, { playerId:
left.playerId }); } // Remove from online users in Redis

```

```

await redis.sRem('online_users', userId).catch(() => {}); // Update
last seen timestamp      await prisma.user.update({      where: { id:
userId },                data: { lastLoginAt: new Date() },    }).catch(() =>
{}); // silently ignore if DB is temporarily busy           // Notify online
friends this user went offline      try {                  const friendships = await
prisma.friend.findMany({      where: {                      OR: [{ requesterId:
userId }, { addresseeId: userId }],                status: 'ACCEPTED',
},                });      const friendIds = friendships.map((f) =>
f.requesterId === userId ? f.addresseeId : f.requesterId    );
if (friendIds.length > 0) {          const allSockets = await
io.fetchSockets();          for (const friendId of friendIds)
{          const friendSocket = allSockets.find((s) => s.data.user?.userId
=== friendId);          friendSocket?.emit(SOCKET_EVENTS.FRIEND_OFFLINE, {
userId, username });          }          } catch { /* non-critical */
}          // Clean up rate limit records for this socket
clearRateLimitEntry(socket.id);
moveRateLimiter.delete(socket.id);    });    }); // end
io.on('connection') }                // end registerSocketHandlers

```

Socket Event Reference — Complete Table

Event (constant)	Direction	Emitter	Receiver	Purpose
AUTH_JOIN	Client → Server	Client (on connect)	Server handler	Enter a room; triggers full room state load
AUTH_ERROR	Server → Client	Server	Joining socket only	Auth or ban error during join
ROOM_STATE	Server → Client	Server	Joining socket only	Full snapshot: players, furniture, chat history
ROOM_PLAYER_JOINED	Server → Room	Server	All existing players	New player arrived; client spawns their avatar
ROOM_PLAYER_LEFT	Server → Room	Server	All remaining players	Player disconnected; client removes their avatar
PLAYER_MOVE	Client → Server	Client (on movement)	Server handler	Client reports new position; server validates & relays
PLAYER_POSITION	Server → Room	Server	All other players	Broadcast validated position update
CHAT_SEND	Client → Server	Client	Server handler	Send a chat

Event (constant)	Direction	Emitter	Receiver	Purpose
				message for filtering and broadcast
CHAT_MESSAGE	Server → Room	Server	All players incl. sender	Confirmed, filtered message broadcast
CHAT_ERROR	Server → Client	Server	Sending socket only	Rate limited or muted notification
AVATAR_UPDATE	Client → Server	Client	Server handler	Outfit/appearance change request
AVATAR_CHANGED	Server → Room	Server	All other players	Broadcast validated avatar change
FRIEND_ONLINE	Server → Friend	Server	Online friends' sockets	Notify friends when this user joins
FRIEND_OFFLINE	Server → Friend	Server	Online friends' sockets	Notify friends when this user disconnects

11.4 Testing the Full Socket Flow

LINUX-1-F4 Start the Server and Verify Socket.io Endpoint LINUX

After the Mac has pushed all socket code and the Oracle VM has pulled, build and start the server. Then use the following sequence to verify the full auth-to-WebSocket pipeline end to end.

Step 1 — Build and Start (on Oracle VM):

```
cd /opt/havenworld/apps/server pnpm build node dist/index.js
```

Step 2 — Verify HTTP health (from Mac Terminal, second window):

```
curl https://[your-ip].nip.io/health # Expected response: #
{"status":"ok", ...}
```

Step 3 — Verify WebSocket in Chrome:

- Open Chrome and navigate to `havenworld.pages.dev`.
- Open DevTools (F12 or `Cmd+Option+I`).
- Register a test account using the game's registration form. Check your email for the Resend verification link and click it.
- Log in with the verified account.
- In DevTools, open the **Network** tab and filter by **WS** (WebSocket).
- You should see an active WebSocket connection to your `nip.io` URL.
- Click the connection and switch to the **Messages** sub-tab.
- You should see outgoing `auth:join` frame followed by an incoming `room:state` frame containing the players, furniture, and chat history arrays.
- If both frames appear, the full real-time pipeline — JWT auth middleware → `auth:join` handler → DB queries → RoomManager → Socket.io room broadcast — is confirmed working.

TROUBLESHOOTING: No WS Connection Visible

If no WebSocket connection appears: (1) Confirm `CORS_ORIGIN` in your `.env` matches `havenworld.pages.dev` exactly. (2) Confirm nginx is proxying `Upgrade: websocket` headers — check the nginx config from Part 4A. (3) Check the server console for an `AUTH_INVALID` log line, which means the client's JWT is not being sent correctly in `socket.handshake.auth.token`.

11.5 Part 4B Completion Checklist

Use this checklist to verify every deliverable in this document before moving on to Part 4C. Each item maps directly to a step ID above. Do not proceed to Part 4C until all boxes are checked.

Auth Routes Group (C1 through C5) — `src/routes/auth.ts`

-
- [] `src/routes/auth.ts` created with register, verify, login, refresh, and logout endpoints
 - [] Zod validation schemas defined for registration (username, email, password, inviteCode) and login (email, password)
 - [] `bcrypt.hash` with 12 rounds used for all passwords — never stored in plain text
 - [] JWT access token (15m expiry) and refresh token (7d expiry) generation working via `generateTokens()`
 - [] Refresh token stored in Redis with 7-day TTL on successful login (`redis.setEx`)
 - [] `httpOnly + Secure + SameSite=Strict` cookie set on login and refresh
 - [] Token refresh endpoint checks Redis allowlist before issuing new token (replay prevention)
 - [] Logout deletes Redis entry immediately and clears the cookie
 - [] Email verification sends HTML email via Resend with signed verification link
 - [] Invite code validated, checked for expiry, and marked as used on successful registration
 - [] Default avatar + personal room created atomically in a single Prisma `$transaction` on registration
 - [] Default free items granted to the new user's inventory via `inventoryService.grantDefaultItems()`

Other Routes Group (C6 through C9) — users, rooms, friends, reports

- [] `src/routes/users.ts` created — endpoints: `me`, `me/avatar`, `me/inventory`, `me/room`, `:username`
- [] `src/routes/rooms.ts` created — endpoints: list public rooms, single room, place furniture, remove furniture
- [] `src/routes/friends.ts` created — endpoints: list friends, incoming requests, send request, accept, delete
- [] `src/routes/reports.ts` created — endpoint: submit player report with enum reason validation
- [] All protected routes use `requireAuth` middleware — unauthenticated requests return 401
- [] All five route files imported and mounted in `src/index.ts` under their correct `/api/...` prefixes

RoomManager Group (D1) — `src/services/RoomManager.ts`

-
- [] `src/services/RoomManager.ts` created and exported as a singleton
 - [] `joinRoom`, `leaveRoom`, `movePlayer`, and `addChatMessage` methods implemented and working
 - [] `setFurniture`, `addFurniture`, and `removeFurniture` methods implemented
 - [] `getOccupantCount()` used by `GET /api/rooms` to inject live player counts into the response
 - [] Personal rooms (`roomId.startsWith('personal-')`) cleaned from memory when last player leaves

Moderation Group (E1) — `src/services/ModerationService.ts`

- [] `src/services/ModerationService.ts` created with no class or database dependency
- [] `moderateMessage()` filters profanity with `bad-words` npm package; returns `ModerationResult`
- [] `checkRateLimit()` enforces per-socket sliding window chat throttle (5 msg / 3 s default)
- [] `clearRateLimitEntry()` called on socket disconnect to prevent memory leak

Socket.io Handlers Group (F1 through F4) — `src/sockets/index.ts`

- [] `src/sockets/index.ts` created and `registerSocketHandlers(io)` called from `src/index.ts`
- [] `socketAuthMiddleware` verifies JWT on every new connection; unauthenticated connections are rejected
- [] `auth:join` loads avatar + furniture from DB (or `RoomManager` cache), joins Socket.io room, emits `room:state`
- [] `auth:join` notifies online friends of the new connection via `friend:online` event
- [] `player:move` validates room membership, coordinate bounds, and rate limit before broadcasting
- [] `chat:send` enforces rate limit + mute check + auto-unmute + profanity filter + DB persist before broadcast
- [] `avatar:update` validates inventory ownership server-side + saves to DB + broadcasts `avatar:changed`
- [] `disconnect` removes player from `RoomManager` + Redis + notifies friends + cleans up both rate limit maps
- [] `pnpm build` completes with zero TypeScript errors

[] Active WebSocket connection visible in Chrome DevTools Network tab with `auth:join` and `room:state` frames confirmed

▶ Continuation — What Comes Next in Part 4C

This is Part 4B. **Part 4C** (the final document in this series) covers:

- **Admin Moderation Routes** — Protected admin endpoints for reviewing reports, muting users, banning accounts, and managing invite codes via the REST API.
- **Invite Code Generation Script** — The CLI script to generate invite code batches and the workflow for distributing them to alpha testers.
- **Production Deployment** — PM2 ecosystem file, startup script, and the complete GitHub Actions SSH deploy workflow that automatically pulls, builds, and restarts the server on every push to `main`.
- **UptimeRobot Monitor Setup** — Configuring uptime monitoring against the `/health` endpoint with alert notifications.
- **Phase 1 Alpha-Ready Gate Checklist** — The final go/no-go checklist covering security, performance, monitoring, and stability criteria that must all be green before the first invite codes are distributed to testers.